Pendule inversé sur Robot Lego NXT

Systèmes Temps Réel

INPT/ENSEEIHT - 3SN E+L

Octobre - Novembre 2025

Plan

1. Préparation de l'environnement de travail
2. Le pendule inversé : un peu de théorie
3. Implantation du pendule inversé sur le robot Lego NXT
4. Travail à effectuer

Plan

1. Préparation de l'environnement de travail
2. Le pendule inversé : un peu de théorie
3. Implantation du pendule inversé sur le robot Lego NXT
4. Travail à effectuer

Solution 1 : J'utilise une station de l'N7

Les étapes :

1. Ajoutez l'environnement Trampoline-OSEK dans votre chemin d'accès :

```
source /mnt/n7fs/nxt/nxt.sh
```

2. Testez si c'est ok : tapez `goil -help`
   Vous devriez avoir la liste des options de la commande `goil`

Remarque

Pour éviter de faire la manip. à chaque fois, vous pouvez ajouter la commande `source /mnt/n7fs/nxt/nxt.sh` à la fin de votre ficher `.bashrc`

Solution 2 : J'utilise ma machine personnelle

Cette solution utilise l'outil `docker` : `https://www.docker.com/`

1. Téléchargez l'archive contenant le Dockerfile et les fichiers script
   - `https://moodle.inp-toulouse.fr/mod/resource/view.php?id=63706`
2. Construisez l'image contenant Trampoline-OSEK pour le NXT :

```
docker build -t trampoline .
```

3. Exécutez le conteneur Docker associé à l'image créée :
   1. Placez vous dans le répertoire dans lequel vous souhaitez travailler
   2. Tapez :

```
docker run -ti \
  --mount src=‘pwd‘,dst=/mnt/shared,type=bind trampoline
```

Remarque :

Le conteneur exécute les outils de développement pour Trampoline. Le répertoire `/mn/shared` est connecté au répertoire courant de la machine hôte. Ainsi tous les fichiers insérés dans ce répertoire seront vus dans le répertoire `/mnt/shared`

Les fichiers fournis

1. Créez le répertoire de travail de votre projet

```
nxt_create nxtSegway
```

   Cette commande crée les fichiers `nxtSegway.c` et `nxtSegway.oil`.

2. Effacez le fichier `nxtSegway.c`.
3. Téléchargez l'archive penduleNXT.tar et décompressez la dans le répertoire de travail nxtSegway :
   - `https://moodle.inp-toulouse.fr/mod/resource/view.php?id=63704`

Fichiers présents dans l'archive :

- `nxt_config.h` : paramètres de configuration du robot Lego NXT
- `tools.c` et `tools.h` : boîte à outils
- `nxtSegway.c` : application du contrôle du robot Lego (à compléter)

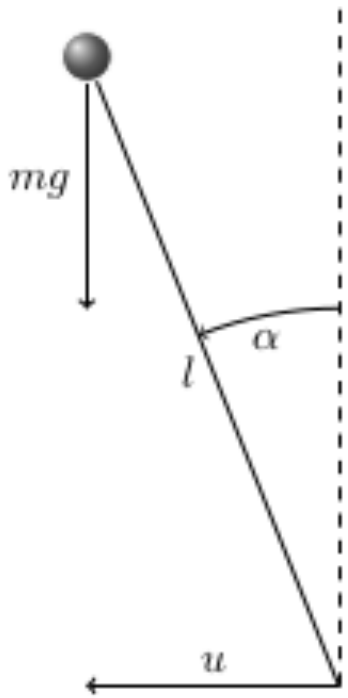
Plan

1. Préparation de l'environnement de travail
2. Le pendule inversé : un peu de théorie
3. Implantation du pendule inversé sur le robot Lego NXT
4. Travail à effectuer

Le pendule inversé : un peu de théorie I

Remarque :

Pour la majorité d'entre vous, vous avez vu cette partie en 1A. L'objectif est de vous faire un petit rappel. Le travail nouveau commence à l'étape 2.



- Un peu de physique, modèle du pendule contrôlé sans frottement :

$$ml^2\ddot{\alpha}(t) + mlg\sin(\alpha(t)) = u(t)$$

- L'état du système s'écrit
  $x(t) = (x_1(t), x_2(t)) = (\alpha(t), \dot{\alpha}(t))$
- Le système différentiel obtenu s'écrit :

$$\begin{cases} \dot{x_1}(t) &= x_2(t) \\ \dot{x_2}(t) &= -\frac{g}{l}\sin(x_1(t)) + \frac{u(t)}{ml^2} \\ x_1(0) &= \alpha_0 \\ x_2(0) &= \dot{\alpha}_0 \end{cases}$$

Le pendule inversé : un peu de théorie II

- Le système devient stable et vertical en appliquant le contrôle $u(t)$ de la façon suivante :

$$u(t) = u_e + K(x(t) - x_e)$$

où $u_e = 0$ et $x_e = 0$ sont les contôles et positions à l'équilibre, K est la matrice de placement de pôle.

- Il est possible d'appliquer une commande en l'ajoutant au contrôle :

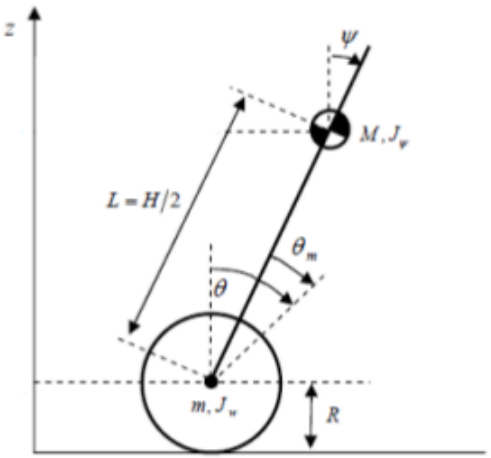
$$z(t) = Ki \int (x(t) - x_o)dt$$

où x_o est l'état objectif.

Et pour le robot Lego ...

- Modèle du robot Lego





- Variable d'état du système

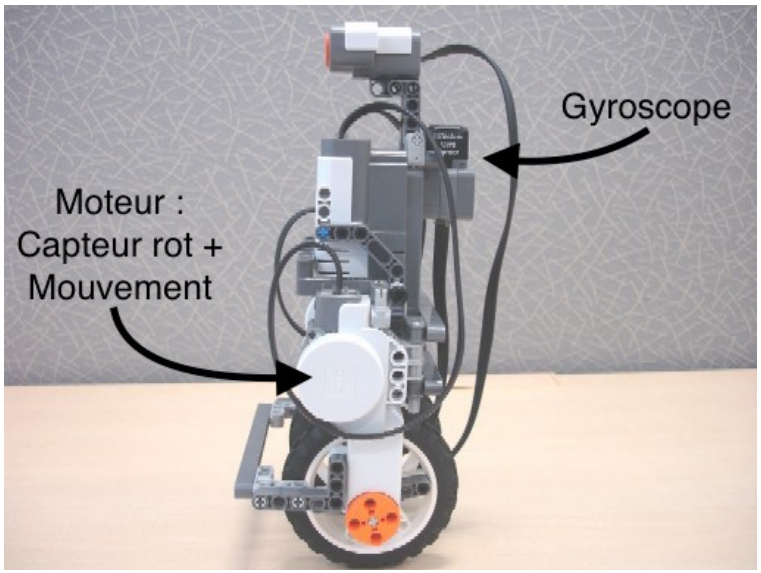
$$x(t) = (\theta, \psi, \dot{\theta}, \dot{\psi})$$

Plan

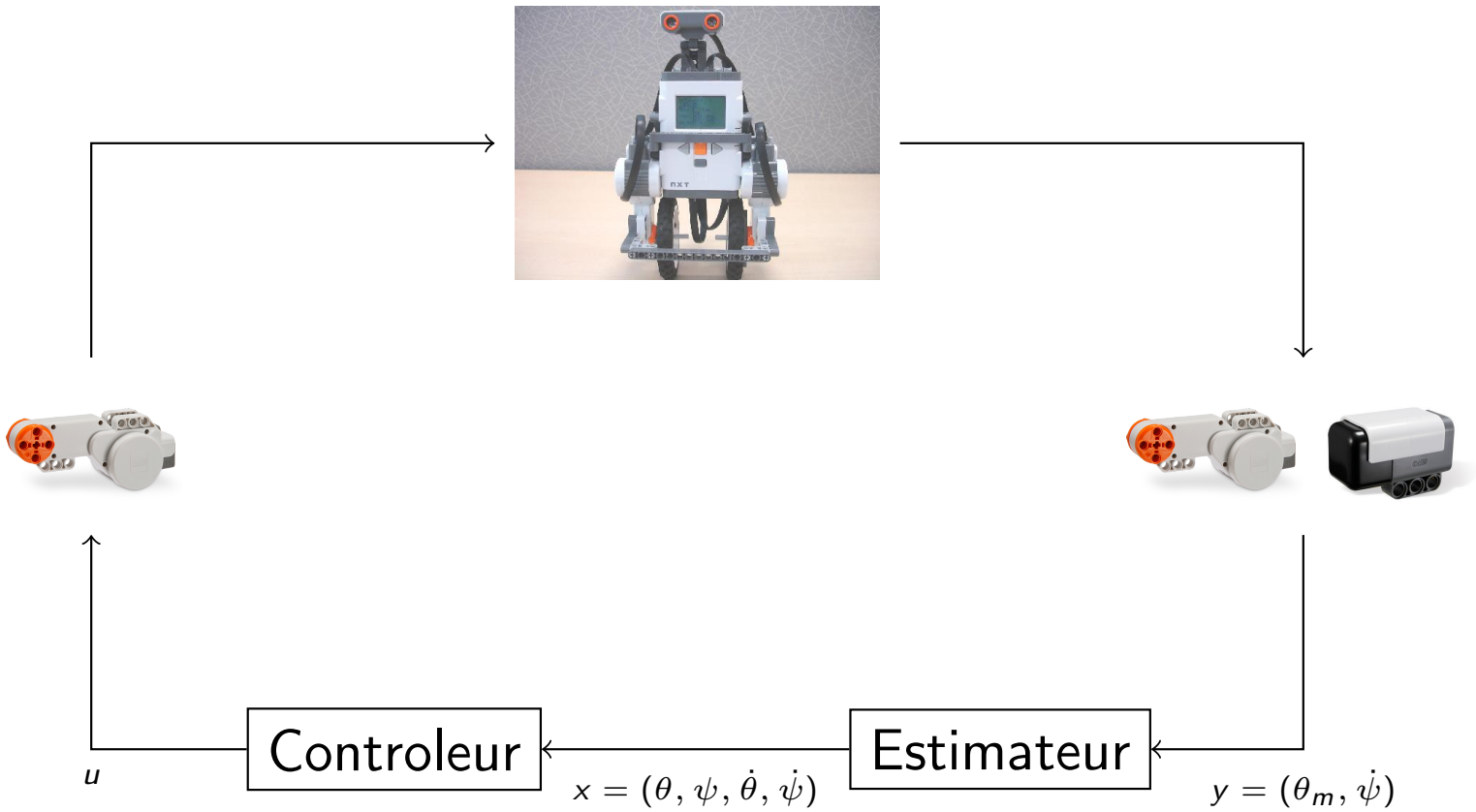
1. Préparation de l'environnement de travail
2. Le pendule inversé : un peu de théorie
3. Implantation du pendule inversé sur le robot Lego NXT
4. Travail à effectuer

Implantation du pendule inversé sur le robot Lego NXT

- Objectifs : comprendre la mise en œuvre du pendule inversé sur le système physique : le robot NXT
- Le robot Lego Mindstorm NXT en configuration pendule inversé :



Le système du pendule inversé



- y : Etat observé du pendule
- x : Etat du pendule
- u : Commande appliquée aux moteurs

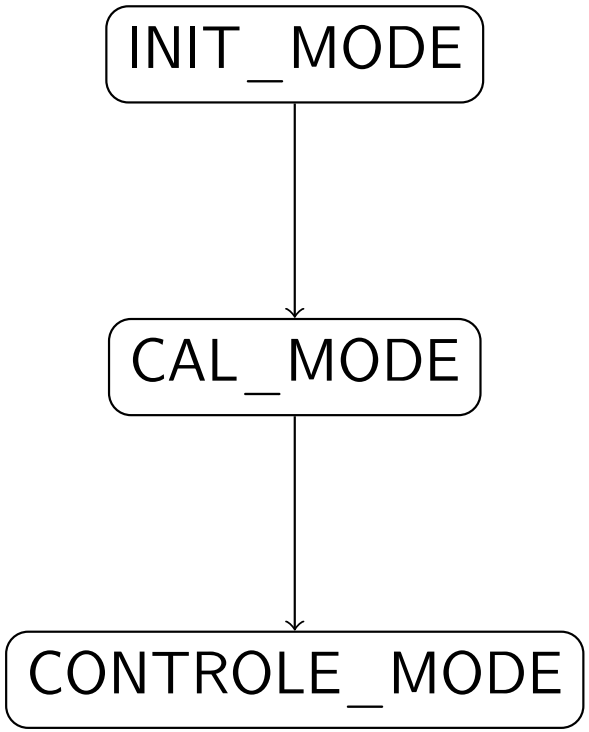
Les outils à disposition

- Le système temps réel OSEK (Trampoline)
  - exécute des tâches de façon périodique
    - ★ les tâches sont exécutées à nouveau après un temps prédéfini
  - Ici, 2 tâches à exécuter : pendule (à compléter), affichage (fournie)
- La boîte à outils
  - La tâche affichage affiche les valeurs de θ et de ψ fournies respectivement par les moteurs et le gyroscope du robot ainsi que l'état courant du robot (valeur de x)
  - Les fonctions :
    - ★ `float getGyro(int gyro_offset)` : fournit en sortie la valeur de $\dot{\psi}$
    - ★ `float getMotorAngle()` : fournit en sortie la rotation des moteurs (i.e. θ_m)
    - ★ `void nxt_motors_set_command(float u)` : applique la commande aux moteurs du robot Lego NXT
    - ★ `float delta_t()` : fournit le pas de discrétisation

La tâche pendule

- Le rôle de cette tâche est de contrôler la position verticale du robot Lego

- 3 états :
  - INIT_MODE : initialise le système (mise à zéro des variables, ...)
  - CAL_MODE :
    - ★ calibre le gyroscope dans la position verticale
    - ★ le robot émet un son lorsqu'effectué
  - CONTROLE_MODE :
    - ★ mode de fonctionnement du pendule inversé
    - ★ c'est la partie à compléter



Plan

1. Préparation de l'environnement de travail
2. Le pendule inversé : un peu de théorie
3. Implantation du pendule inversé sur le robot Lego NXT
4. Travail à effectuer

Etape 1 : Mise en œuvre de l'application

1. Créez la fonction `estimateur` qui fournit l'état courant du système $x = (\theta, \psi, \dot{\theta}, \dot{\psi})$ à partir de l'état observé $y = (\theta_m, \dot{\psi})$ et du pas de discrétisation, avec $\theta = \theta_m + \psi$
2. Créez la fonction `controleur` qui calcule la commande u à appliquer au système en fonction de l'état courant du système
3. Complétez l'état CONTROLE_MODE de la tâche `pendule` contrôlant le mouvement du robot

Rappel intégration/dérivée discrète (Méthode de Euler)

- Intégration : $f(x_i) = f(x_{i-1}) + (x_i - x_{i-1})f'(x_i)$
- Dérivée : $f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$

où $x_i - x_{i-1}$ est le pas de discrétisation.

Valeurs de K et Ki

K=(0.6700,19.9053,1.0747,1.9614)
Ki=0.2534

Etape 2 : Configuration de OSEK et compilation

1. Modifiez le fichier `nxtSegway.oil` de façon à configurer les tâches `pendule` (période = 4 ms) et `affichage` (période = 500 ms)
   Le timer du système a une période de 1 ms.
2. Compilez et exécutez le programme sur le robot Lego
   - Générez un fichier Makefile à partir du fichier `.oil` présent dans l'archive :
     `nxt_goil nxtSegway.oil`
   - Compilation :
     `make`
   - Téléchargement sur le robot :
     `nxt_send nxtSegway_exe.rxe`

Etape 3 : Détection d'un obstacle

1. Ecrivez une nouvelle tâche qui permet l'acquisition d'une distance à l'aide du capteur ultrason et fournit une commande au robot le faisant reculer. La période de cette nouvelle tâche est de 40 ms.
2. Modifiez le fichier `nxtSegway.oil` en conséquence
3. Testez la nouvelle application sur le robot Lego

- Le capteur ultrason doit être initialisé :
  `void ecrobot_init_sonar_sensor(int Input_Port)`
- Acquisition de la distance par le capteur ultrason :
  `int ecrobot_get_sonar_sensor(int Input_Port)`
- Port auquel est connecté le capteur : `PORT_SONAR`

Etape 4 : Initialisation du système à l'aide d'une tâche

1. Ecrivez une nouvelle tâche qui initialise le système et lance la tâche `pendule` par envoi d'un événement
2. Rajoutez dans le fichier `nxtSegway.oil` la configuration de l'événement
3. Testez la nouvelle application sur le robot Lego

Etape 5 : Initialisation du système via les Hook Routines

- La routine StartupHook permet d'effectuer des opérations au démarrage d'OSEK

1. Ecrivez le code de la routine StartupHook qui initialise le système
   `void StartupHook(void)`
2. Activez l'utilisation de la routine StartupHook dans le fichier `nxtSegway.oil`
3. Testez la nouvelle application sur le robot Lego